

TD 7 - Terminaison

Contexte

- ★ il y a N sites qui ont tous des noms distincts, et savent que leurs noms sont distincts. De plus, N est connu de tous les sites.
- ★ les communications sont fiables : tout message émis est délivré au bout d'un temps arbitraire mais fini ;
- ★ les liaisons sont bidirectionnelles, le graphe de communication est fortement connexe ;

Exercice(s)

Exercice 1 – L'algorithme de Huang (1988)

Cet algorithme généralise l'algorithme de Rana (1980) et permet de détecter la terminaison sur une topologie de réseau quelconque.

On distingue les messages de l'application des messages de l'algorithme de détection de terminaison (messages de *contrôle*).

Les sites disposent de trois primitives de communication :

- `send(<type, contenu, horloge, emetteur, recepateur>)` : envoie un message de l'émetteur au récepteur ;
- `recv(<type, contenu, horloge, emetteur, recepateur>)` : réception d'un message envoyé par l'émetteur ;
- `bcast(<type, -, horloge, emetteur, ->)` : permet à l'émetteur de diffuser une information (de contrôle) à tous les autres sites.

Notes :

- Le champ *contenu* n'est pas utilisé lors de l'envoi et réception d'un message de contrôle, il n'est utile que pour les messages de l'application.
- Pour les messages d'application, *type* = *mes*. Pour les messages de contrôle, *type* peut être égale à *term* (annonce de terminaison) ou *agr* (réponse à un message *term*). De plus, on pourra définir d'autres types pour les messages contrôle.
- les messages de l'application ne portent pas d'horloge, que les messages de contrôle.

L'algorithme repose sur l'utilisation d'horloges logiques scalaires. Chaque site i gère une horloge $hfin_i$ lui permettant de mémoriser la date de fin supposée de l'application. Cette date est mise à jour :

- chaque fois que i détecte sa terminaison locale : il est *inactif* et aucun des messages qu'il a envoyés n'est en transit ; cette condition peut être réalisée plusieurs fois si le site est réactivé en recevant un message de l'application ;
- lorsqu'un autre site lui annonce sa terminaison, si l'horloge portée par le message est postérieure à la date de fin qu'il connaît. Pour avoir un ordre total sur les horloges, on utilise le couple (h, id) . Chaque site doit mémoriser donc l'identité (*id*) du site qui, à sa connaissance, s'est terminé le dernier (*dernier_i*).

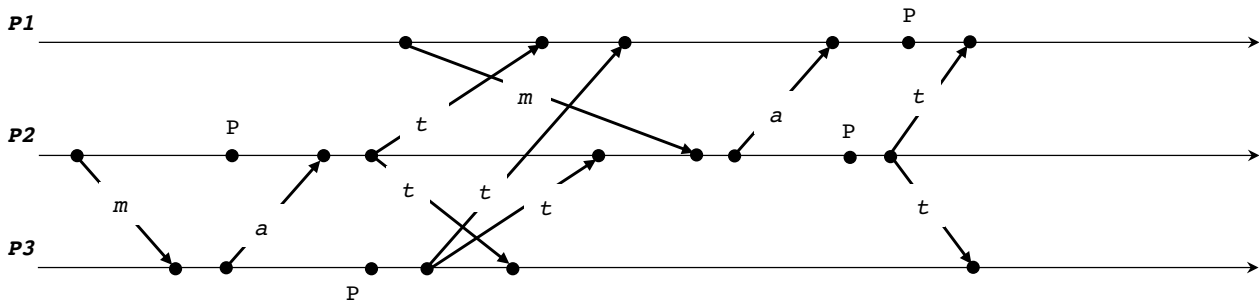


FIGURE 1 – Exécution sur 3 sites

Le principe est le suivant :

- un site i qui détecte sa terminaison locale incrémente son horloge h_i et diffuse l'information sur sa terminaison (message *term*) à tous les autres sites ;
- un site qui reçoit une annonce de terminaison met à jour son horloge et ne répond (envoi d'un message *agr*) que s'il est "localement terminé" et si l'horloge contenue dans le message est supérieure à son horloge locale ;

Question 1

A quel moment un site peut-il décider que l'application est terminée?

Question 2

Afin de contrôler les messages d'application en transite, proposez un mécanisme permettant à chaque site de savoir si un message envoyé a été reçu. Donnez les variables nécessaires à la gestion de ce mécanisme.

Question 3

En considérant que la variable $etat_i$ sauvegarde l'état du processus i (soit *actif* soit *inactif*) et qu'un processus *inactif* en recevant un message de l'application devient *actif*, donnez les pseudocodes des deux primitives suivantes exécutées par le site i :

- *envoyer*($\langle \text{contenu}, i, j \rangle$) : i envoie un message de l'application à j .
- *recevoir*($\langle \text{contenu}, j, i \rangle$) : i reçoit un message de l'application de j .

Question 4

A quel moment le site i peut-il lancer une détection de la terminaison ?

Donnez le pseudocode des primitives exécutées par le site i , (1) lorsqu'il passe de l'état *actif* à l'état *inactif*, et (2) lorsqu'il reçoit un acquittement.

Question 5

Donnez le pseudocode de la primitive exécutée par le site i lorsqu'il reçoit un message de type `term`.

Question 6

Donnez le pseudocode de la primitive exécutée par le site i lorsqu'il reçoit un message de type `agr`.

Question 7

Complétez le schéma d'exécution de la Figure 1 en ajoutant les messages manquants et les horloges des messages (les états P représentent les états où un site devient inactif, les lettres indiquent les types de messages : m pour les messages de l'application, a pour ack et t pour term).

Question 8

Quelles sont les modifications à apporter afin d'optimiser l'algorithme sont organisés en anneau et les messages (*application* et *contrôle*) ne circulent que sur l'anneau?